



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

180+ Important ASP.NET Core interview questions

ASP.NET Core Middleware Pipeline

1. What is middleware in ASP.NET Core?

Middleware is a component in the **HTTP request pipeline** that can:

- Handle requests,
- Pass requests to the next middleware,
- Or short-circuit the pipeline.

Middleware can:

- Perform actions **before** and/or **after** the next middleware executes.
- Be used for logging, authentication, error handling, etc.

Middleware executes in the order it's added in **Program.cs**.



2. How is the middleware pipeline configured (in `Program.cs` / `Startup.cs`)?

- In ASP.NET Core **6+** (minimal hosting model), middleware is added in `Program.cs`:

```
var builder = WebApplication.CreateBuilder(args);  
var app = builder.Build();  
  
app.UseMiddleware<YourMiddleware>();  
app.UseRouting();  
app.UseEndpoints(endpoints => { endpoints.MapControllers();  
});  
  
app.Run();
```

- In older versions (e.g., .NET Core 3.1), `Startup.cs` is used:

```
public void Configure(IApplicationBuilder app,  
IWebHostEnvironment env)  
{  
    app.UseMiddleware<YourMiddleware>();  
    app.UseRouting();
```



```
app.UseEndpoints(endpoints => {  
    endpoints.MapControllers();  
});
```

3. Difference between `app.Use`, `app.UseMiddleware`, `app.Run`, and `app.Map`

Method	Description
<code>app.Use</code>	Adds middleware that can call next in the pipeline.
<code>app.UseMiddleware<T>()</code>	Adds a custom middleware class .
<code>app.Run</code>	Terminal middleware – does not call next . Ends the pipeline.
<code>app.Map</code>	Branches the pipeline based on URL path (e.g. <code>/api</code>).

Example:

```
app.Use(async (context, next) => {  
    await next(); // go to next middleware  
});
```

```
app.Run(async context => {
```



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

```
        await context.Response.WriteAsync("Hello World"); //
terminates pipeline
});
```

4. How do you write a custom middleware?

Create a class with:

- A constructor accepting `RequestDelegate`
- An `Invoke` or `InvokeAsync` method

```
public class MyCustomMiddleware
{
    private readonly RequestDelegate _next;

    public MyCustomMiddleware(RequestDelegate next) => _next =
next;

    public async Task InvokeAsync(HttpContext context)
    {
        // Pre-processing logic
        await _next(context);
        // Post-processing logic
    }
}
```

Follow on LinkedIn: [Sandeep Pal](https://www.linkedin.com/in/sandeepall)



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

```
}
```

Register it:

```
app.UseMiddleware<MyCustomMiddleware>();
```

5. What is `RequestDelegate`?

`RequestDelegate` is a **delegate** representing the next middleware in the pipeline:

```
public delegate Task RequestDelegate(HttpContext context);
```

In custom middleware, it allows passing control to the next component.

6. Order of middleware: why it matters?

Middleware is **executed in the order it's added**, and this order affects behavior.

Example:

```
app.UseAuthentication(); // Must come before authorization
app.UseAuthorization();
```

```
app.UseEndpoints(...);
```

Follow on LinkedIn: [Sandeep Pal](#)



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

Logging, error handling, and security middlewares must be **early** in the pipeline.

7. How do you short-circuit the pipeline?

Simply **don't call** `await next()` in a middleware:

```
if (!context.User.Identity.IsAuthenticated)
{
    context.Response.StatusCode = 401;
    return; // Short-circuits
}
await next(); // only called if authenticated
```

8. Use of `UseStaticFiles` and its place in pipeline

`app.UseStaticFiles()` enables serving files from `wwwroot`.

Important: It must be added **before routing or endpoints** so static files are served without invoking controller logic.

```
app.UseStaticFiles();
app.UseRouting();
app.UseEndpoints(...);
```

Follow on LinkedIn: [Sandeep Pal](#)



9. How to handle exceptions using middleware

(UseExceptionHandler, UseDeveloperExceptionPage)

- `UseDeveloperExceptionPage()` shows detailed errors (development only).
- `UseExceptionHandler("/Error")` handles errors in production with a custom page or handler.

Example:

```
if (env.IsDevelopment())
    app.UseDeveloperExceptionPage();
else
    app.UseExceptionHandler("/Error");
```

Or inline:

```
app.UseExceptionHandler(errorApp =>
{
    errorApp.Run(async context =>
    {
        context.Response.StatusCode = 500;
        await context.Response.WriteAsync("An error
occurred");
    });
});
```



```
});  
});
```

10. How to enable HTTPS redirection middleware

Redirects HTTP requests to HTTPS.

```
app.UseHttpsRedirection();
```

Add **early** in the pipeline, before auth or routing.

You can configure HTTPS ports in `launchSettings.json` or `appsettings.json`.

11. How do you serve static files with custom file providers or options (like caching, directory browsing)?

```
app.UseStaticFiles(new StaticFileOptions  
{  
    FileProvider = new  
PhysicalFileProvider(Path.Combine(env.ContentRootPath,  
"MyFiles")),  
    RequestPath = "/Files",  
    OnPrepareResponse = ctx =>  
    {
```



```
        ctx.Context.Response.Headers.Append("Cache-Control",  
        "public,max-age=600");  
    }  
});
```

For **directory browsing**:

```
app.UseDirectoryBrowser(new DirectoryBrowserOptions  
{  
    FileProvider = new PhysicalFileProvider("path"),  
    RequestPath = "/browse"  
});
```

12. Differences between terminal middleware vs non-terminal middleware

Type	Description
Terminal	Ends the pipeline. Doesn't call <code>next()</code> . E.g., <code>app.Run()</code>
Non-Termi nal	Calls <code>next()</code> and allows other middlewares to run after it. E.g., <code>app.Use()</code>

Terminal middleware:

```
app.Run(async ctx => {  
    await ctx.Response.WriteAsync("This ends the pipeline");  
});
```



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

```
});
```

13. How to integrate authentication/authorization middleware

```
app.UseAuthentication(); // Validates user identity
app.UseAuthorization();  // Applies policies/roles
```

Order matters: must be **after routing** but **before endpoints**.

```
app.UseRouting();
app.UseAuthentication();
app.UseAuthorization();
app.UseEndpoints(...);
```

14. How to enable and configure CORS via middleware

```
builder.Services.AddCors(options =>
{
    options.AddPolicy("MyPolicy", policy =>
    {
        policy.WithOrigins("https://example.com")
            .AllowAnyHeader()
            .AllowAnyMethod();
    });
});
```

Follow on LinkedIn: [Sandeep Pal](https://www.linkedin.com/in/sandeepal)



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

```
app.UseCors("MyPolicy");
```

Must be placed **before routing/endpoints**.

15. Using middleware for logging or performance (e.g. measuring execution time)

Example custom middleware to measure time:

```
public class TimingMiddleware
{
    private readonly RequestDelegate _next;

    public TimingMiddleware(RequestDelegate next) => _next =
next;

    public async Task InvokeAsync(HttpContext context)
    {
        var sw = Stopwatch.StartNew();
        await _next(context);
        sw.Stop();

        Console.WriteLine($"Request took
{sw.ElapsedMilliseconds} ms");
    }
}
```

Follow on LinkedIn: [Sandeep Pal](#)



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

```
}  
  
}
```

Register:

```
app.UseMiddleware<TimingMiddleware>();
```

Dependency Injection (DI)

16. What is Dependency Injection and why is it used?

Dependency Injection (DI) is a design pattern that allows you to inject dependencies (services) into classes instead of hard-coding them.

- ◆ Promotes **loose coupling**
- ◆ Enhances **testability**
- ◆ Encourages **separation of concerns**

17. ASP.NET Core's built-in DI container: capabilities and limitations

✅ Capabilities:

- Constructor injection
- Lifetime management (Transient, Scoped, Singleton)
- Supports `IEnumerable<T>`, `IServiceProvider`, and open generics

⚠️ Limitations:

Follow on LinkedIn: [Sandeep Pal](#)



- No support for named registrations
- Limited property injection
- Basic feature set compared to Autofac or other 3rd-party containers

18. Different service lifetimes: Transient, Scoped, Singleton

Lifetime	Description	Use Case Example
Transient	New instance every time	Lightweight stateless services
Scoped	One instance per request	Database context, UoW
Singleton	One instance for the app's lifetime	Logging, Config access

19. How to register services in ConfigureServices

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddTransient<IMyService, MyService>();
    services.AddScoped<IRepository, Repository>();
    services.AddSingleton<ILoggerService, LoggerService>();
}
```

20. How to resolve dependencies in controllers, razor pages, middleware

- **Controllers:** Constructor injection
- **Razor Pages:** Constructor injection in `PageModel`
- **Middleware:** Inject via constructor or use `ApplicationBuilder.ApplicationServices`



21. Constructor injection vs property injection (and which is supported)

- **Constructor Injection:** Supported and preferred in ASP.NET Core.
- **Property Injection:** Not supported natively in built-in DI; can be achieved via custom logic or 3rd-party containers.

✓ Use constructor injection for immutability and clarity.

22. Using IServiceProvider / IServiceScopeFactory

- **IServiceProvider:** Resolves services manually.
- **IServiceScopeFactory:** Creates a new DI scope (useful for background tasks).

```
using (var scope = serviceScopeFactory.CreateScope())
{
    var scopedService =
scope.ServiceProvider.GetRequiredService<IMyScopedService>();
}
```

23. How scoped services behave in background tasks vs request scope

- Scoped services are tied to the **HTTP request** lifecycle.
- In **background tasks**, you must **manually create a scope** using **IServiceScopeFactory** to resolve scoped services safely.

24. How to override default DI behavior (e.g., remove or replace services)



To replace:

```
services.AddSingleton<IService, CustomImplementation>();
```

To remove:

```
var descriptor = services.First(x => x.ServiceType ==  
typeof(IMyService));  
services.Remove(descriptor);
```

25. Using Options pattern (**IOptions**, **IOptionsSnapshot**, **IOptionsMonitor**)

- **IOptions<T>**: For singleton/config services.
- **IOptionsSnapshot<T>**: Scoped; updates per request.
- **IOptionsMonitor<T>**: Singleton; can react to config changes.

```
services.Configure<MySettings>(Configuration.GetSection("MySettings")  
));
```

26. When is **IHostedService** / **BackgroundService** used and how DI works there

- Use **IHostedService** or **BackgroundService** for tasks that run in the background.
- Services are injected via constructor. Scoped services must use **IServiceScopeFactory**.

27. Injecting the configuration provider (**IConfiguration**)

Follow on LinkedIn: [Sandeep Pal](#)



`IConfiguration` is automatically registered and injected via constructor.

```
public MyService(IConfiguration config)
{
    var key = config["MyKey"];
}
```

28. Injecting logging (`ILogger<T>`)

ASP.NET Core provides structured logging via `ILogger<T>`.

```
public class MyService
{
    private readonly ILogger<MyService> _logger;
    public MyService(ILogger<MyService> logger) => _logger = logger;

    public void DoSomething() => _logger.LogInformation("Action
performed");
}
```

29. Handling circular dependencies

Occurs when two services depend on each other directly or indirectly.

🔧 Fix:

- Refactor to remove circular reference
 - Use `Lazy<T>` or factory injection
 - Split responsibilities
-

30. How to mock dependencies in unit tests



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

Use a mocking framework like **Moq**:

```
var mockService = new Mock<IMyService>();  
mockService.Setup(s => s.Get()).Returns("test");  
  
var controller = new MyController(mockService.Object);
```

Mocking helps isolate the unit of work and test behavior independently.

MVC & Razor Pages

31. What is MVC in ASP.NET Core? How does it differ from MVC in “older” ASP.NET?

- **MVC (Model-View-Controller)** is a pattern for organizing code into:
 - **Model:** Data and business logic
 - **View:** UI rendering
 - **Controller:** Input handling and app flow

◆ Key differences in ASP.NET Core:

- Built on a **modular, cross-platform** framework.



- Uses **dependency injection by default**.
- **Unified Web API + MVC pipeline**.
- Lightweight and **middleware-based** request processing.

32. What is Razor Pages? When to use Razor Pages instead of MVC?

- Razor Pages is a **page-based programming model** introduced in ASP.NET Core 2.0.
- Each **.cshtml** file has an associated **PageModel** class (like a view + controller combined).
- Best for **simple UI-focused apps** or **CRUD** pages.

✓ Use Razor Pages for:

- Simpler, page-focused apps

✓ Use MVC for:

- Complex, controller-based routing or **large, structured apps**
-



33. How routing works in MVC / Razor Pages

- **MVC Routing:**

- Uses **attribute routing** or **conventional routing**.

Example:

```
endpoints.MapControllerRoute(  
    name: "default",  
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

○

- **Razor Pages Routing:**

- Based on **folder structure**.
- URL `/Products/Edit` maps to `/Pages/Products/Edit.cshtml`.

34. What are controllers, actions, views, and view components

- **Controller:** Handles HTTP requests.



- **Action Method:** A method in the controller that returns a result (like a view or JSON).
- **View:** `.cshtml` file that renders HTML.
- **View Component:** Reusable mini-views with logic (like partials but with code-behind).

35. Using Tag Helpers vs HTML Helpers

Tag Helpers: Use HTML-like syntax. Easier to read/maintain.

```
<form asp-controller="Home" asp-action="Login"></form>
```

-

HTML Helpers: C# methods used in Razor.

```
@Html.BeginForm("Login", "Home")
```

-

✓ Prefer Tag Helpers in modern ASP.NET Core apps.

36. ViewData, ViewBag, TempData: differences and uses



Type	Lifetime	Dynami c?	Purpose
ViewDa ta	Current request	No	Pass data to views
ViewBa g	Current request	Yes	ViewData wrapper (dynamic)
TempD ata	Across requests	No	Preserved for 1 redirect only

37. Model Binding in MVC/Razor Pages

- Automatically maps **form values, query strings, route data** to C# model properties.

```
public IActionResult Submit(User user) { ... }
```

- Binds complex types and simple types out-of-the-box.

38. Model Validation using Data Annotations

Decorate model properties:

```
public class User {
```



```
[Required]
[EmailAddress]
public string Email { get; set; }
}
```

- Automatically validated during model binding.
- Use `ModelState.IsValid` to check.

39. Custom Validation (IValidatableObject, Custom Validators)

- **IValidatableObject:**

```
public IEnumerable<ValidationResult>
Validate(ValidationContext context)
```

- **Custom Attribute:**

```
public class MyCustomAttribute : ValidationAttribute {
    public override bool IsValid(object value) { ... }
}
```



40. Layouts, Partial Views, View Components

- **Layouts:** Shared structure (e.g. header, footer) using `_Layout.cshtml`.
- **Partial Views:** Reusable UI snippets (`_LoginPartial.cshtml`).
- **View Components:** Partial views with logic.

```
public class CartViewComponent : ViewComponent {  
    public IViewComponentResult Invoke() => View("Cart",  
model);  
}
```

41. How to pass data from controller to view

- **Via ViewBag / ViewData / Model:**

```
return View(model); // Strongly typed  
ViewBag.Message = "Hello";  
ViewData["Count"] = 5;
```

42. Filters: Action filters, result filters, exception filters



- **ActionFilter:** Runs before/after action method.
- **ResultFilter:** Runs before/after view result.
- **ExceptionHandler:** Catches unhandled exceptions.
- Apply globally or via attributes.

```
public class LogActionFilter : IActionFilter { ... }
```

43. Razor Page Handlers (OnGet, OnPost, etc.)

- Razor Pages use handler methods:

```
public class IndexModel : PageModel {  
    public void OnGet() { ... }  
    public IActionResult OnPost() { ... }  
}
```

- You can use `asp-page-handler="Update"` for custom methods.
-

44. Dependency Injection in Razor Pages



- Inject services into `PageModel` constructor:

```
public IndexModel(IMyService service) { ... }
```

- Also available in Razor views via `@inject`:

```
@inject ILogger<MyPage> Logger
```

45. Razor Class Libraries (RCL)

- Reusable libraries that contain **Razor views, pages, static files, etc.**
- Share UI components across multiple projects.

```
dotnet new razorclasslib
```

46. Areas in MVC

- Used to **organize large applications** into sections (e.g., Admin, Customer).
- Each Area has its own **Controllers/Views/Models**.



```
[Area("Admin")]  
public class DashboardController : Controller { ... }
```

47. Bundling, Minification, Static Assets Organization

- Handled via:
 - **ASP.NET Core Middleware**
 - **WebOptimizer, Gulp, or Webpack**
- Static files in `wwwroot/`
- Enable in `Startup.cs`:

```
app.UseStaticFiles();
```

48. Supporting multiple view engines / customizing them

- ASP.NET Core supports **Razor** by default.
- You can add support for **custom view engines** by implementing `IViewEngine`.



49. Localization / Globalization in views

- Use `IStringLocalizer<T>` or `IViewLocalizer`:

```
@inject IViewLocalizer Localizer
<h1>@Localizer["Welcome"]</h1>
```

- Add resource `.resx` files for each language.

50. Razor Pages vs MVC: Performance Considerations

- **Performance:** Very similar, both use Razor rendering.
- **Simplicity:** Razor Pages have less boilerplate for page-based UIs.
- **Maintainability:** Razor Pages better for small apps, MVC better for large, modular apps.

Web API (RESTful Services)

Web API (RESTful Services)



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

51. What is REST? How to design RESTful APIs in ASP.NET Core

REST (Representational State Transfer) is an architectural style for building scalable web services. RESTful APIs follow standard HTTP methods (GET, POST, PUT, DELETE) and stateless communication.

✓ Key principles for RESTful API in ASP.NET Core:

- Use **HTTP verbs** for operations.
- Use **nouns** (resources) in URLs, not verbs.
- Support **stateless operations**.
- Return **appropriate HTTP status codes**.
- Implement **HATEOAS** (optional for hypermedia).

52. What is the [ApiController] attribute and what benefits it brings?

The [ApiController] attribute is used to denote Web API controllers in ASP.NET Core.

✓ Benefits:

- **Automatic model validation**



- Infer `[FromBody]` / `[FromQuery]`, etc.
- **400 BadRequest** returned automatically if model state is invalid.
- Improved parameter binding behavior

53. Routing conventions in Web API (attribute routing, route templates)

- **Attribute Routing (Preferred):**

```
[Route("api/products")]
[ApiController]
public class ProductsController : ControllerBase {
    [HttpGet("{id}")]
    public IActionResult Get(int id) { ... }
}
```

- **Route Templates:**

Use placeholders like `{id}`, constraints like `{id:int}`.

You can also define route prefixes at controller level and use relative routes in actions.



54. How to version Web APIs (URL versioning, header versioning, media type versioning)

Use **Microsoft.AspNetCore.Mvc.Versioning** package.

✓ Supported methods:

- **URL versioning:** `/api/v1/products`
- **Header versioning:** `X-API-Version: 1.0`
- **Media Type versioning:** `Accept: application/vnd.company.v1+json`

```
services.AddApiVersioning(options => {  
    options.ReportApiVersions = true;  
    options.AssumeDefaultVersionWhenUnspecified = true;  
    options.DefaultApiVersion = new ApiVersion(1, 0);  
});
```

55. Content negotiation (JSON, XML)

ASP.NET Core selects the response format based on the **Accept** header.



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET,CLOUD,API & REACT)

@Sandeep_pall

- JSON is the default.
- To add XML:

```
services.AddControllers()  
    .AddXmlSerializerFormatters();
```

```
Accept: application/xml
```

56. Binding parameters: from body, from query, from route, from form

Source	Attribute
Body	[FromBody]]
Query string	[FromQuer y]
Route	[FromRout e]
Form data	[FromForm]



```
Header    [FromHeader  
          ]
```

ASP.NET Core infers the source when possible.

57. Handling large uploads/downloads (streaming, chunking)

- For large **uploads**, use `IFormFile`, `Stream`, or read from `Request.Body` for streaming.
- For **downloads**, return `FileStreamResult`.

Enable buffering or streaming to avoid memory overload.

58. Error handling / global exception handling in APIs

Use `UseExceptionHandler` middleware or custom `ExceptionHandler`.

You can also create a **global error handler**:

```
app.UseExceptionHandler(config => {  
    config.Run(async context => {  
        // Log and return problem details  
    });  
});
```



Or use **ProblemDetails** for structured error responses.

59. Returning appropriate HTTP status codes

Use proper status codes:

- 200 OK (success)
- 201 Created (on POST)
- 204 No Content (on DELETE)
- 400 Bad Request
- 401 Unauthorized / 403 Forbidden
- 404 Not Found
- 500 Internal Server Error

```
return Ok(result);  
return NotFound();  
return BadRequest(ModelState);
```

60. Using **ActionResult<T>** vs **ActionResult**

Follow on LinkedIn: [Sandeep Pal](#)



- **ActionResult<T>**: Combines a **return value** and **response code**.

```
public ActionResult<Product> Get(int id) { ... }
```

- **IActionResult**: More flexible, returns different result types explicitly.

Prefer **ActionResult<T>** for simpler, strongly-typed APIs.

61. Asynchronous APIs (async/await)

ASP.NET Core supports full async/await pattern for IO-bound tasks.

[HttpGet]

```
public async Task<ActionResult<Product>> GetAsync(int id) {  
    var product = await _repo.GetAsync(id);  
    return product == null ? NotFound() : Ok(product);  
}
```

- ✓ Improves scalability and performance.
-

62. CORS for Web APIs

Enable CORS (Cross-Origin Resource Sharing) to allow client apps from different domains:

Follow on LinkedIn: [Sandeep Pal](https://www.linkedin.com/in/sandeepall)



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

```
services.AddCors(options => {
    options.AddPolicy("AllowFrontend", builder =>
        builder.WithOrigins("https://frontend.com")
            .AllowAnyHeader()
            .AllowAnyMethod());
});

app.UseCors("AllowFrontend");
```

63. Rate limiting / throttling

ASP.NET Core doesn't include rate limiting out of the box (pre-.NET 8).

Use libraries like:

- **AspNetCoreRateLimit**
- **YARP, Azure API Management**

.NET 8 introduced built-in **RateLimiterMiddleware**.

64. API documentation: Swagger / OpenAPI

Use **Swashbuckle.AspNetCore** or **NSwag** for generating Swagger docs.

```
services.AddSwaggerGen();
```

Follow on LinkedIn: [Sandeep Pal](#)



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

```
app.UseSwagger();  
app.UseSwaggerUI();
```

Supports:

- Auto-generated OpenAPI docs
- Try it out interface
- Versioning support

65. Testing Web API endpoints (unit / integration)

- **Unit testing:** Mock services/repositories using **Moq** or **FakeItEasy**.
- **Integration testing:** Use `WebApplicationFactory<T> + TestServer + HttpClient`.

```
var client = _factory.CreateClient();  
var response = await client.GetAsync("/api/products");
```

66. Authorization/authentication in APIs (JWT, OAuth)

Follow on LinkedIn: [Sandeep Pal](#)



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

- Use **JWT Bearer Tokens** for stateless auth:

```
services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options => { ... });
```

- Secure endpoints with:

```
[Authorize(Roles = "Admin")]
```

- OAuth supported via **IdentityServer**, **Azure AD**, or **external providers**.

67. Securing APIs: Anti-Forgery, CORS, HTTPS, etc.

- **HTTPS:** Enforced via `UseHttpsRedirection()`
- **CORS:** Limit origins using policies
- **Anti-forgery:** Usually not needed for APIs unless using cookies (use `ValidateAntiForgeryToken`)



- Use **authentication** + **authorization** checks for all endpoints
-

68. DTOs and mapping (e.g. AutoMapper)

- DTOs decouple internal models from API contracts.
- Use **AutoMapper** for mapping:

```
CreateMap<Product, ProductDto>();
```

```
var dto = _mapper.Map<ProductDto>(product);
```

69. Versioning pitfalls and backward compatibility

- Avoid **breaking changes** to existing endpoints.
 - Use **new versions** for changes in contracts.
 - Maintain old versions as long as needed.
 - Ensure documentation and clients are updated accordingly.
-



70. Handling concurrency / optimistic concurrency

- Use **RowVersion** / **ETag** to detect concurrent edits.

Example with EF Core:

```
modelBuilder.Entity<Product>()  
    .Property(p => p.RowVersion).IsRowVersion();
```

Return **409 Conflict** if concurrency exception is caught.

Model Binding & Validation

71. How model binding works: what sources it considers

Model binding in ASP.NET Core maps incoming HTTP data to C# parameters or model properties.

Sources considered:

- Route data
- Query string
- Form data
- Headers
- JSON body (**application/json**)



- Uploaded files
- Services (via `[FromServices]`)

ASP.NET Core **automatically binds** data based on parameter types and attributes (`[FromBody]`, `[FromQuery]`, etc.).

72. Binding complex types vs simple types

- **Simple types** (int, string, bool, DateTime, etc.): Bound from route, query string, or form fields.
- **Complex types** (custom classes): Bound from multiple sources like query/form fields or JSON body.

```
public IActionResult Create([FromBody] Product product)
```

73. Custom model binders

Use a **custom model binder** when default binding doesn't work (e.g., custom formats, headers).

```
public class CustomBinder : IModelBinder {  
    public Task BindModelAsync(ModelBindingContext context) {  
        // Custom logic here  
    }  
}
```

Register with `[ModelBinder]` or globally in `Startup`.

74. Handling multiple binder source attributes (`[FromBody]`, `[FromQuery]`, etc.)



You cannot bind multiple `[FromBody]` parameters in a single action.

Common sources:

- `[FromBody]` — for JSON/XML payloads
- `[FromQuery]` — query string
- `[FromRoute]` — route values
- `[FromForm]` — form fields and file uploads
- `[FromHeader]` — HTTP headers

75. Model validation: data annotations

Decorate models with attributes:

```
public class User {  
    [Required]  
    [StringLength(50)]  
    [EmailAddress]  
    public string Email { get; set; }  
}
```

Used in both **MVC** and **API** for validation.

76. Server-side validation vs client-side validation (unobtrusive)

- **Server-side:** Always performed on the server; use `ModelState.IsValid`.
- **Client-side:** HTML5 + jQuery Unobtrusive Validation (for MVC/Razor Pages).

Client-side validation **improves UX**, but server-side is **essential for security**.



77. Custom validation attributes

Create custom rules by inheriting `ValidationAttribute`:

```
public class MustBeEvenAttribute : ValidationAttribute {  
    public override bool IsValid(object value) {  
        return (int)value % 2 == 0;  
    }  
}
```

Use like:

```
[MustBeEven]  
public int Number { get; set; }
```

78. `IValidatableObject` interface

Use for **cross-field validation** within a model:

```
public class Product : IValidatableObject {  
    public string Name { get; set; }  
    public decimal Price { get; set; }  
  
    public IEnumerable<ValidationResult> Validate(ValidationContext  
context) {  
        if (Price < 0) {  
            yield return new ValidationResult("Price must be  
positive");  
        }  
    }  
}
```



79. Using FluentValidation

- Install `FluentValidation.AspNetCore`
- Create a validator class:

```
public class UserValidator : AbstractValidator<User> {  
    public UserValidator() {  
        RuleFor(x => x.Email).NotEmpty().EmailAddress();  
    }  
}
```

Register with:

```
services.AddFluentValidationAutoValidation();
```

✅ Offers **more readable** and **testable** validation logic than data annotations.

80. Validation in API vs MVC (error response formatting)

- `[ApiController]` handles automatic validation:
 - Returns `400 BadRequest` with `ValidationProblemDetails` if `ModelState.IsValid`.
- In MVC (without `[ApiController]`), you need to manually check `ModelState.IsValid`.

```
if (!ModelState.IsValid) return View(model);
```

81. Model state: checking `ModelState.IsValid`

Used to **check if bound model passed validation**:

Follow on LinkedIn: [Sandeep Pal](#)



```
if (!ModelState.IsValid) {  
    return BadRequest(ModelState);  
}
```

MVC automatically adds errors to `ModelState` based on validation attributes.

82. Binding nested objects and collections

ASP.NET Core supports **binding nested properties**:

```
public class Order {  
    public Customer Customer { get; set; }  
    public List<Product> Products { get; set; }  
}
```

Works seamlessly from JSON or form data if property names match.

83. Handling missing or invalid data

- Use `[Required]` for non-nullable fields.
 - For optional values, use nullable types.
 - Use `ModelState` to report and handle missing/invalid fields.
 - Return custom error messages if needed.
-

84. Sanitization of input

Model binding **does not sanitize input** — it binds raw data.

🛡️ To prevent attacks (XSS, injection), sanitize:



- Strings: HTML encode output (@Html.Encode)
- Manually clean input before use
- Use **antivirus/malware scanners** for uploaded files

85. Binding of files (IFormFile)

Used for **file uploads** from forms (not [FromBody]):

```
public IActionResult Upload(IFormFile file)
{
    var path = Path.Combine("uploads", file.FileName);
    using var stream = new FileStream(path, FileMode.Create);
    file.CopyTo(stream);
}
```



For multiple files:

```
List<IFormFile> files
```

Configuration & AppSettings

86. Role of appsettings.json, appsettings.Development.json, etc.

- **appsettings.json**: Base configuration file used across all environments.
- **appsettings.{Environment}.json**: Environment-specific overrides (e.g., Development, Production).



ASP.NET Core loads them automatically based on the environment:



`ASPNETCORE_ENVIRONMENT=Development`

✅ Loaded in order of precedence, where later files override earlier ones.

87. Environment-based configuration (Development, Staging, Production)

ASP.NET Core uses the `ASPNETCORE_ENVIRONMENT` variable to determine the runtime environment.

Supported environments (by convention):

- Development
- Staging
- Production

Environment-specific logic can be applied:

```
if (env.IsDevelopment()) { ... }
```

Also used to load:

- `appsettings.{env}.json`
 - `Startup{env}.cs` (in older versions)
-

88. Overriding settings via environment variables, command line

✅ ASP.NET Core supports **hierarchical override** of config sources:

- Environment variables override `appsettings.json`:



```
MyApp__Logging__LogLevel__Default=Warning
```

- Command line arguments override everything:

```
dotnet run --Logging:LogLevel:Default=Debug
```

89. Using IConfiguration to read settings

Inject IConfiguration anywhere:

```
public class MyService {  
    private readonly string _apiKey;  
    public MyService(IConfiguration config) {  
        _apiKey = config["MySettings:ApiKey"];  
    }  
}
```

You can also access nested settings via `config.GetSection("MySettings")`.

90. Binding configuration sections to POCOs

Bind config to strongly typed objects:

```
public class MySettings {  
    public string ApiKey { get; set; }  
    public int Timeout { get; set; }  
}
```

```
services.Configure<MySettings>(config.GetSection("MySettings"));
```

Use via `IOptions<MySettings>`.



91. Using Options pattern (`IOptions<T>`, `IOptionsSnapshot<T>`, `IOptionsMonitor<T>`)

- `IOptions<T>`: Reads settings once at startup.
- `IOptionsSnapshot<T>`: Gets updated settings per request (for scoped services).
- `IOptionsMonitor<T>`: Supports **change notifications** and works in singleton services.

```
public MyService(IOptions<MySettings> options) {  
    var settings = options.Value;  
}
```

92. Secret management (e.g. user secrets, Azure Key Vault)

- **User Secrets** (for local dev):

```
dotnet user-secrets init  
dotnet user-secrets set "MySettings:ApiKey" "secret"
```

- **Azure Key Vault:**

```
builder.Configuration.AddAzureKeyVault(...);
```

✓ Secure sensitive data like API keys, connection strings, tokens.

93. Logging configuration: how to configure log levels

In `appsettings.json`:

```
"Logging": {
```



```
"LogLevel": {  
  "Default": "Information",  
  "Microsoft": "Warning",  
  "Microsoft.Hosting.Lifetime": "Information"  
}  
}
```

Supports built-in providers: Console, Debug, EventSource, Azure, etc.

Custom configuration through `ILogger<T>`.

94. Connection strings management

Stored under "ConnectionStrings" section in `appsettings.json`:

```
"ConnectionStrings": {  
  "DefaultConnection":  
  "Server=.;Database=AppDb;Trusted_Connection=True;"  
}
```

Read via:

```
var conn = config.GetConnectionString("DefaultConnection");
```

Or inject via Options pattern.

95. Settings reloading (on change) if supported

JSON files support live reload:

```
builder.Configuration.AddJsonFile("appsettings.json", optional:  
false, reloadOnChange: true);
```



`IOptionsMonitor<T>` automatically updates bound values when config changes.

 **Does not work with all sources** (e.g., env vars, command line).

96. Strongly typed configuration

Using POCOs and Options pattern allows **type-safe config**:

```
services.Configure<MySettings>(config.GetSection("MySettings"));
```

Use `[Required]`, `[Range]`, etc., to add validation.

97. Validating configuration (e.g., using `IValidateOptions`)

You can validate bound config using `IValidateOptions<T>`:

```
public class MySettingsValidator : IValidateOptions<MySettings> {
    public ValidateOptionsResult Validate(string name, MySettings
options) {
        if (string.IsNullOrEmpty(options.ApiKey)) {
            return ValidateOptionsResult.Fail("ApiKey is
required.");
        }
        return ValidateOptionsResult.Success;
    }
}
```

Register with DI:

```
services.AddSingleton<IValidateOptions<MySettings>,
MySettingsValidator>();
```

98. Configuration providers (JSON, XML, INI, Environment, Azure, etc.)

Follow on LinkedIn: [Sandeep Pal](https://www.linkedin.com/in/sandeepall)



ASP.NET Core supports multiple configuration providers:

- JSON (default)
- Environment Variables
- Command Line Args
- INI files
- XML files
- In-memory
- Azure App Configuration
- Azure Key Vault
- Secrets Manager

Each can be chained with priority.

99. Default values and optional settings

Use default values in POCOs or fallback logic:

```
public class MySettings {  
    public string ApiKey { get; set; } = "default-api-key";  
}
```

Or:

```
var apiKey = config["MySettings:ApiKey"] ?? "fallback";
```

100. Masking sensitive configuration data



✓ Best practices:

- **Don't log sensitive values**
- Use `[JsonIgnore]` or remove values before logging
- Mask in logs manually:

```
_logger.LogInformation("Token: {Token}", Mask(token));
```

- **Never store secrets in source-controlled files** (`appsettings.json`)
- Use **user-secrets**, **Key Vault**, or **environment variables** instead

Authentication & Authorization (JWT, Identity)

101. What is authentication vs authorization?

- **Authentication:** Verifying the identity of a user (Who are you?)
- **Authorization:** Determining if the authenticated user has permission to perform an action (What are you allowed to do?)

In ASP.NET Core, both are handled via middleware and attributes like `[Authorize]`, roles, and policies.

102. How to configure ASP.NET Core Identity



ASP.NET Core Identity provides a complete solution for:

- User registration & login
- Roles, claims, tokens
- Password hashing
- Two-factor authentication

✓ Setup:

```
dotnet add package Microsoft.AspNetCore.Identity.EntityFrameworkCore
```

In `Startup` or `Program.cs`:

```
services.AddIdentity<IdentityUser, IdentityRole>()  
    .AddEntityFrameworkStores<ApplicationDbContext>()  
    .AddDefaultTokenProviders();
```

103. JWT Bearer tokens: what it is, how to configure

JWT (JSON Web Token) is a compact, URL-safe token format used for authentication.

✓ Configure JWT auth:

```
services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)  
    .AddJwtBearer(options => {  
        options.TokenValidationParameters = new  
TokenValidationParameters {  
            ValidateIssuer = true,  
            ValidateAudience = true,  
            ValidateLifetime = true,  
            ValidateIssuerSigningKey = true,  
            IssuerSigningKey = new  
SymmetricSecurityKey(Encoding.UTF8.GetBytes("your-secret"))  
        };  
    });
```

Follow on LinkedIn: [Sandeep Pal](#)



```
});
```

Use `[Authorize]` to secure endpoints.

104. Role-based vs policy-based authorization

✓ Role-based:

```
[Authorize(Roles = "Admin")]
```

✓ Policy-based:

```
services.AddAuthorization(options => {  
    options.AddPolicy("CanEdit", policy =>  
        policy.RequireClaim("EditPermission"));  
});
```

Then use:

```
[Authorize(Policy = "CanEdit")]
```

Policy-based gives more flexibility (custom requirements, claims, logic).

105. Using Claims

Claims are user attributes (e.g., email, role, permissions).

Add claims when creating identity:

```
var claims = new List<Claim> {  
    new Claim(ClaimTypes.Name, user.UserName),  
    new Claim("Permission", "Edit")  
};
```



Access in code:

```
var claim = User.FindFirst("Permission)?.Value;
```

106. Cookie-based authentication

Used in traditional MVC apps for session-based auth.

```
services.AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme)
    .AddCookie(options => {
        options.LoginPath = "/Account/Login";
    });
```

On login:

```
await HttpContext.SignInAsync(principal);
```

Cookies are stored in the browser and sent with each request.

107. External logins (OAuth, OpenID Connect)

Use built-in providers:

```
services.AddAuthentication()
    .AddGoogle(options => {
        options.ClientId = "...";
        options.ClientSecret = "...";
    });
```

Also supports:

- Facebook



- Microsoft
- Twitter
- OpenID Connect
- Azure AD

Use `RemoteAuthenticationHandler<T>` or Identity scaffolding.

108. Refresh tokens

Used with **JWT** to renew access tokens after expiration without logging in again.

- Issue refresh token along with access token.
- Store securely (DB or secure HTTP-only cookie).
- On access token expiration, send refresh token to get a new one.

You must manually implement refresh token logic (not built-in to Identity).

109. Token expiration, token revocation

- JWTs typically expire in 5–30 minutes.
- Expired tokens cannot be used.
- **Revocation** requires token blacklisting (e.g., database of revoked tokens).

✅ Configure expiration:

```
Expires = DateTime.UtcNow.AddMinutes(30)
```

✅ Use refresh tokens to handle expiration.

Follow on LinkedIn: [Sandeep Pal](#)



110. Securing routes with `[Authorize]` and `[AllowAnonymous]`

- `[Authorize]`: Requires authenticated user
- `[Authorize(Roles = "Admin")]`: Requires role
- `[AllowAnonymous]`: Allows access without login

Example:

```
[Authorize]
public IActionResult Dashboard() { }
```

```
[AllowAnonymous]
public IActionResult Login() { }
```

111. Custom authorization policies and handlers

Define complex authorization rules using **IAuthorizationHandler**:

```
public class MinimumAgeRequirement : IAuthorizationRequirement {
    public int Age { get; }
    public MinimumAgeRequirement(int age) => Age = age;
}

public class MinimumAgeHandler :
    AuthorizationHandler<MinimumAgeRequirement> {
    protected override Task HandleRequirementAsync(...) {
        // logic
    }
}
```

Register in DI and use with `[Authorize(Policy = "...")]`.



112. Data protection (for cookies, tokens)

ASP.NET Core uses **Data Protection API** to:

- Encrypt authentication cookies
- Protect tokens (e.g., password reset tokens)

Configure:

```
services.AddDataProtection()  
    .PersistKeysToFileSystem(new DirectoryInfo("path"))  
    .SetApplicationName("AppName");
```

Used internally by Identity and cookie middleware.

113. Multi-tenant auth scenarios

Multi-tenancy can involve:

- Per-tenant databases
- Per-tenant user stores
- Per-tenant roles/policies

Strategies:

- Use claims to store tenant ID
- Filter data per tenant
- Custom middleware for tenant resolution

Can integrate with IdentityServer4 or Azure AD B2C for federated auth.

Follow on LinkedIn: [Sandeep Pal](#)



114. Securing password storage

ASP.NET Core Identity uses **PBKDF2** hashing by default.

Best practices:

- Never store plaintext passwords
- Use salted + hashed storage
- Use `PasswordHasher<T>` or Identity defaults

Use:

```
PasswordHasher<T>.HashPassword(user, password)
```

Can switch to Argon2, Bcrypt via custom password hasher.

115. Two-factor authentication (2FA)

Supported out of the box in ASP.NET Core Identity.

Options:

- SMS (via provider)
- Email
- Authenticator app (TOTP)

Enable via Identity configuration:

```
services.Configure<IdentityOptions>(options => {  
    options.SignIn.RequireConfirmedEmail = true;  
});
```



Use `SignInManager<T>` to handle verification and token generation.

Filters & Middleware Overlaps

116. What are filters in ASP.NET Core?

Filters are components that run code before or after certain stages in the request pipeline within MVC or Razor Pages. Types of filters include:

- **Authorization filters:** Run before any other filter to check user authorization.
- **Resource filters:** Run after authorization and before model binding.
- **Action filters:** Run before and after the execution of action methods.
- **Exception filters:** Handle unhandled exceptions thrown by actions.
- **Result filters:** Run before and after action results are executed.

Filters allow you to inject logic at these specific points.

117. How do filters differ from middleware?

- **Middleware** is part of the global HTTP request pipeline and executes for every request.
- **Filters** only run within MVC/Razor pipeline, for controller actions or Razor Pages handlers.
- Middleware runs earlier and can short-circuit the entire pipeline; filters run later, scoped to MVC processing.



- Middleware is good for cross-cutting concerns affecting all requests, filters are better for concerns around controller/action execution.

118. What is the order of filters?

Filters run in a specific order depending on their type:

- Authorization filters run first.
- Resource filters run next.
- Model binding happens after resource filters.
- Action filters run around the action execution.
- Exception filters handle exceptions thrown during action or result execution.
- Result filters run around result execution.

Within each type, filters can be ordered by their **Order** property and whether they are global, controller-level, or action-level.

119. How to create and use custom filters?

Create a custom filter by implementing one of the filter interfaces like:

```
public class CustomActionFilter : IActionFilter
{
    public void OnActionExecuting(ActionExecutingContext context)
    {
        // Before action executes
    }
    public void OnActionExecuted(ActionExecutedContext context)
    {
        // After action executes
    }
}
```



```
}  
}
```

Register globally in `Startup`:

```
services.AddControllersWithViews(options =>  
{  
    options.Filters.Add<CustomActionFilter>();  
});
```

Or decorate controllers/actions:

```
[ServiceFilter(typeof(CustomActionFilter))]  
public class HomeController : Controller { ... }
```

120. Global filters vs per-controller/action filters

- **Global filters** apply to all MVC actions, registered via `AddControllers` or `AddMvc` options.
- **Controller or action filters** are applied via attributes directly on controllers or actions.
- Global filters are best for cross-cutting concerns like logging, exception handling.
- Controller/action filters are best for behavior specific to particular routes or endpoints.

121. What is filter context and what can filters do?

Filters receive context objects (e.g., `ActionExecutingContext`) providing:

- HTTP context and request data.



- Access to action parameters.
- Ability to modify or cancel execution (e.g., short-circuit).
- Access to the result or exceptions.
- Ability to set result or modify response.

This allows filters to inspect, modify, or block processing at their stage.

122. What is short-circuiting in filters?

Filters can **short-circuit** by setting the result early, preventing further execution:

```
public void OnActionExecuting(ActionExecutingContext context)
{
    if (!IsAuthorized())
    {
        context.Result = new UnauthorizedResult(); // stops pipeline
here
    }
}
```

This prevents action execution and later filters from running.

123. Filter attributes vs service-based filters

- **Filter attributes** are instantiated per request and support parameters, but have limited DI capabilities.
- **Service-based filters** (using `ServiceFilter` or `TypeFilter`) allow filters to be resolved from DI container, enabling constructor injection.

Use service-based filters when you need dependencies injected.



124. Combining filters with middleware for cross-cutting concerns

- Use **middleware** for concerns that affect all requests (logging, CORS, authentication).
- Use **filters** for MVC-specific concerns tied to action execution (authorization, validation, caching).
- Filters can complement middleware for granular control within MVC.
- Example: Use middleware for global exception logging, filters for handling MVC-specific exceptions and returning appropriate views or API responses.

Versioning, CORS

125. Implementing API versioning

API versioning enables multiple versions of your API to coexist, allowing clients to migrate gradually.

Common ways to version APIs in ASP.NET Core:

- **URL versioning** (e.g., `/api/v1/products`)
- **Query string versioning** (e.g., `/api/products?api-version=1.0`)
- **Header versioning** (custom header like `api-version: 1.0`)
- **Media type versioning** (via `Accept` header, e.g., `application/json;v=1`)

Use the `Microsoft.AspNetCore.Mvc.Versioning` NuGet package:

```
services.AddApiVersioning(options => {  
    options.AssumeDefaultVersionWhenUnspecified = true;  
});
```



```
options.DefaultApiVersion = new ApiVersion(1, 0);  
options.ReportApiVersions = true;  
});
```

126. Semantic versioning / version negotiation

Semantic versioning (semver) uses MAJOR.MINOR.PATCH format, e.g., 1.2.0.

- **MAJOR** version changes break backward compatibility.
- **MINOR** versions add functionality in a backward-compatible manner.
- **PATCH** versions are for backward-compatible bug fixes.

Version negotiation allows clients and servers to agree on an API version via headers or URL. Servers should support multiple versions and respond with supported version info.

127. Deprecation strategies

- Mark old versions as deprecated via documentation and HTTP response headers.
 - Return warning headers or custom fields indicating version deprecation.
 - Gradually phase out old versions, allowing clients to migrate.
 - Consider introducing sunset policies and endpoints to notify clients.
-

128. CORS: What is it?

CORS (Cross-Origin Resource Sharing) is a browser security feature that restricts web pages from making requests to a different domain than the one that served the web page, to prevent cross-site attacks.



CORS defines a way for servers to allow controlled access to resources from a different origin.

129. How to set CORS policies in ASP.NET Core

Configure in `Startup.cs` or `Program.cs`:

```
services.AddCors(options =>
{
    options.AddPolicy("AllowSpecificOrigin",
        builder =>
        {
            builder.WithOrigins("https://example.com")
                .AllowAnyHeader()
                .AllowAnyMethod();
        });
});
```

Enable middleware:

```
app.UseCors("AllowSpecificOrigin");
```

130. Preflight requests

- For certain CORS requests (e.g., methods other than GET/POST or custom headers), browsers send an **OPTIONS** request first, called a **preflight**.
- The server must respond with allowed methods, headers, and origins.
- Properly configured CORS policies handle preflight requests automatically.

131. Configuring CORS globally vs per endpoint



- **Global CORS:** Apply a policy for all endpoints by adding middleware early in the pipeline with `app.UseCors(...)`.
- **Per-endpoint CORS:** Apply CORS policies selectively using the `[EnableCors("PolicyName")]` or `[DisableCors]` attributes on controllers or actions.

132. Handling cross-origin credentials

- To allow cookies or credentials in cross-origin requests, configure:

```
builder.WithOrigins("https://example.com")
    .AllowCredentials()
    .AllowAnyHeader()
    .AllowAnyMethod();
```

- Clients must send requests with `credentials: 'include'`.
- Note: Allowing credentials disables the wildcard (*) origin.

133. Security implications of CORS

- Improperly configured CORS can expose your API to CSRF and data theft.
- Avoid using `AllowAnyOrigin` with `AllowCredentials` as browsers block it.
- Restrict origins to trusted domains.
- Validate CORS headers and avoid overly permissive policies.
- Use HTTPS to secure cross-origin requests.



Cross-Cutting / Advanced / “Miscellaneous”

134. Hosting: Kestrel, IIS, reverse proxy scenarios

- **Kestrel** is the default cross-platform web server for ASP.NET Core, lightweight and fast.
 - **IIS** acts as a reverse proxy on Windows, forwarding requests to Kestrel.
 - Reverse proxies improve security, manage SSL, handle load balancing.
 - On Linux, Nginx or Apache often act as reverse proxies to Kestrel.
-

135. InProcess vs OutOfProcess hosting

- **InProcess hosting** runs ASP.NET Core app inside the IIS worker process (`w3wp.exe`), better performance.
 - **OutOfProcess hosting** runs the app in a separate process, IIS proxies requests to it.
 - InProcess is default in ASP.NET Core 3.0+ for IIS hosting.
-

136. Health checks

- Health checks provide endpoints to report app health.
- Use `Microsoft.AspNetCore.Diagnostics.HealthChecks`.
- Configure checks for databases, external services, dependencies.
- Useful for Kubernetes, load balancers.



137. Logging: built-in logging, third party (Serilog, NLog)

- ASP.NET Core has built-in logging with providers (Console, Debug, EventSource).
- Third-party libs like **Serilog** and **NLog** offer rich sinks, structured logging.
- Configure logging via `appsettings.json` or code.

138. Performance tuning: caching (in memory, distributed), response compression

- **In-memory caching** stores data on server memory for fast retrieval.
- **Distributed caching** uses external stores (Redis, SQL) for multiple servers.
- **Response compression** reduces payload size using gzip, Brotli middleware.

139. Threading concerns, concurrency, deadlocks

- Avoid blocking calls in async code to prevent deadlocks.
- Use `async/await` properly.
- Protect shared data with locks or concurrent collections.
- Avoid thread starvation with thread pool tuning.

140. Memory leaks, disposal of dependencies

- Memory leaks can occur if `IDisposable` objects aren't disposed.



- Use dependency injection lifetimes properly.
- Be cautious with static references and events holding objects.
- Use tools like dotMemory, PerfView.

141. Scalability: load balancing, statelessness

- Design apps to be **stateless** so any server instance can handle requests.
- Use distributed caches or external session stores.
- Load balancers distribute traffic to multiple instances.

142. Deployment: Docker, Azure, CI/CD pipelines

- Containerize apps with **Docker** for consistent deployments.
- Use Azure App Services, Azure Kubernetes Service for cloud hosting.
- Set up CI/CD pipelines (GitHub Actions, Azure DevOps) for automated builds and deployments.

143. Monitoring, metrics, tracing (OpenTelemetry etc)

- Monitor app health, request metrics, errors.
 - Use **OpenTelemetry** for distributed tracing.
 - Integrate with Application Insights, Prometheus, Grafana.
-



144. Security best practices (HTTPS, XSS, CSRF, OWASP)

- Enforce **HTTPS**.
 - Use **Anti-forgery tokens** to prevent CSRF.
 - Sanitize input to prevent **XSS**.
 - Follow **OWASP** guidelines to secure apps.
-

145. Middleware for compression, caching, etc.

- Use **Response Compression Middleware** to compress responses.
 - Use **Response Caching Middleware** to cache GET responses.
 - Middleware can be composed for layered processing.
-

146. Response caching, output caching

- **Response caching** caches HTTP responses based on headers.
 - **Output caching** (new in ASP.NET Core 7) caches entire output of endpoints.
-

147. Distributed caching / session state

- Use Redis or SQL Server for distributed cache/session in multi-server environments.
 - Helps maintain session state without sticky sessions.
-



148. SignalR (real-time communication)

- Enables real-time bi-directional communication.
 - Supports WebSockets, Server-Sent Events, long polling.
-

149. Background tasks / hosted services

- Use `IHostedService` or `BackgroundService` to run background tasks.
 - Useful for timers, queue processing.
-

150. Working with files and streaming

- Stream large files efficiently using `FileStreamResult`.
 - Use async IO to avoid blocking.
-

151. Version of .NET Core / .NET changes (what's new in latest)

- Track latest improvements like minimal APIs, source generators, performance boosts.
 - Keep updated with latest SDKs.
-

Scenario / Design & Best Practices

152. How to structure a large ASP.NET Core solution (layers, clean architecture, onion architecture)



- Use layered architecture: Presentation (API/UI), Application (services, business logic), Domain (entities, business rules), Infrastructure (data access, external services).
- **Clean Architecture** emphasizes separation of concerns and dependency direction towards the domain layer.
- **Onion Architecture** structures the app around the core domain, with dependencies pointing inward.
- Use projects to separate concerns and improve maintainability.

153. Dependency inversion, SOLID principles in ASP.NET Core

- Apply **Dependency Inversion Principle** by coding against abstractions (interfaces), not implementations.
- Follow **SOLID** principles:
 - Single Responsibility: Each class has one reason to change.
 - Open/Closed: Classes open for extension, closed for modification.
 - Liskov Substitution: Subtypes can replace base types.
 - Interface Segregation: Use multiple specific interfaces.
 - Dependency Inversion: Depend on abstractions.

154. Handling resource cleanup (e.g. DbContext disposal)

- Use DI to manage lifecycle of `DbContext` as Scoped.
- Avoid manual disposal; rely on DI container.



- Dispose IDisposable objects promptly when created manually.
- Use `using` statements for short-lived resources.

155. Design for testability (unit tests, integration tests)

- Write loosely coupled code via DI.
- Mock external dependencies for unit tests.
- Use in-memory databases (e.g., InMemory EF Core) for integration tests.
- Isolate layers to enable focused testing.

156. Logging vs tracing vs exception reporting

- **Logging:** recording app events/info.
- **Tracing:** tracking execution flow across components or services.
- **Exception reporting:** capturing and notifying on errors.
- Use structured logging and distributed tracing for diagnostics.

157. Data access: EF Core best practices (no tracking, lazy loading, migrations)

- Use **No Tracking** queries for read-only data to improve performance.
- Use lazy loading cautiously; prefer explicit loading to avoid surprises.
- Manage migrations with proper version control.



- Use async queries to avoid blocking.

158. Handling database migrations in production safely

- Apply migrations in CI/CD pipeline with proper backups.
- Use transactional migrations where supported.
- Run migrations during maintenance windows.
- Test migrations in staging before production.

159. Versioning and backward compatibility of APIs

- Maintain multiple API versions with clear deprecation policies.
- Avoid breaking changes; use additive changes.
- Communicate version lifecycle clearly to consumers.

160. Rate limiting / throttling strategies

- Protect APIs from abuse using rate limits (requests per second/minute).
- Use libraries like **AspNetCoreRateLimit** or API Gateway features.
- Implement IP-based or user-based throttling.

161. Error / exception handling patterns



- Use global exception handling middleware.
 - Return meaningful HTTP status codes and error details.
 - Avoid leaking sensitive info.
 - Implement retry policies for transient errors.
-

162. Graceful shutdown

- Handle shutdown signals to complete ongoing requests.
 - Dispose resources correctly.
 - Use `IHostApplicationLifetime` events to hook into shutdown process.
-

163. Handling secrets securely

- Use **Azure Key Vault**, **AWS Secrets Manager**, or **HashiCorp Vault**.
 - Avoid storing secrets in code or config files.
 - Use environment variables or user secrets during development.
-

164. Use of configuration and environment variables in CI/CD

- Inject config via environment variables or secret stores in pipelines.
- Use multiple config files per environment (`appsettings.Development.json`).
- Secure sensitive values using CI/CD secret management.



165. Internationalization / localization

- Use resource files (.resx) for strings.
- Configure `RequestLocalizationMiddleware`.
- Support multiple cultures and fallback cultures.
- Localize data formats, dates, currencies.

166. Validation of inputs to avoid security vulnerabilities (e.g. SQL injection, XSS etc)

- Use parameterized queries / ORM to prevent SQL injection.
- Sanitize and encode user input to prevent XSS.
- Validate inputs rigorously on server side.
- Use built-in validation attributes and custom validators.

Sample / Misc Interview Questions

167. What is the difference between `IActionResult` and `ActionResult<T>`?

- **`IActionResult`**: Represents a non-generic result of an action method. Can return any HTTP response (Ok, NotFound, Redirect, etc.).
- **`ActionResult<T>`**: Combines a result with a typed value (`T`). It allows returning typed data or an HTTP response. Improves clarity and enables better OpenAPI docs.



168. What is the role of Startup class now with newer versions of .NET vs minimal hosting model?

- In .NET 5 and earlier, **Startup** configures services and middleware.
- In .NET 6+ minimal hosting model, the **Program.cs** file combines service registration and middleware setup with a simplified, top-level statement style.
- **Startup** can still be used for organization, but not required.

169. How to migrate an ASP.NET Core application from .NET 5 to .NET 8 (or newer)?

- Update target framework in project file (**net8.0**).
- Update NuGet packages and dependencies.
- Adapt code for minimal APIs if desired.
- Replace **Startup** with minimal hosting model if preferred.
- Test thoroughly for API changes or obsoleted APIs.
- Review and update middleware and routing patterns.

170. What is minimal API and how is it different from controllers?

- Minimal APIs are lightweight endpoints defined with top-level statements, no controller classes.
- Ideal for microservices or simple APIs.
- Less ceremony and fewer files.



- Controllers provide richer features (filters, model binding, action results).

171. How do you handle version conflicts (e.g., multiple dependencies needing different versions of a library)?

- Use **binding redirects** (in .NET Framework).
- Use **assembly version unification** and strong-named assemblies.
- In .NET Core, resolve via **NuGet package management**, use **central package versions**.
- Consider upgrading or consolidating dependencies.

172. What is endpoint routing?

- Introduced in ASP.NET Core 3.0.
- Centralized routing system that decouples route matching from middleware.
- Routes requests to endpoints defined by controllers, Razor Pages, minimal APIs.
- Supports route-based middleware filters.

173. Difference between conventional routing and attribute routing

- **Conventional routing:** Routes defined centrally (usually in `Startup`), patterns applied globally.
- **Attribute routing:** Routes declared directly on controllers/actions via attributes (`[Route]`, `[HttpGet]`).



- Attribute routing is more flexible and explicit.

174. How to enable OpenAPI / Swagger UI and customizing it

- Add `Swashbuckle.AspNetCore` NuGet package.
- Configure Swagger services (`builder.Services.AddSwaggerGen()`).
- Enable middleware (`app.UseSwagger()`, `app.UseSwaggerUI()`).
- Customize with options like API info, document filters, UI themes.

175. What is content negotiation, how ASP.NET Core chooses media types

- Automatically selects response format (JSON, XML) based on `Accept` header.
- Configured via formatters in MVC options.
- Falls back to default formatter if no match.

176. What is ProblemDetails in WebAPI

- Standardized error response format defined by RFC 7807.
 - Contains properties like `status`, `title`, `detail`, `instance`.
 - Used by default in ASP.NET Core for error responses.
-



177. How to return custom error responses / use custom middleware for error responses

- Implement custom exception handling middleware.
 - Catch exceptions, set HTTP status, and return custom JSON payload.
 - Use `UseExceptionHandler()` or global filters.
 - Customize `ProblemDetails` or your own error models.
-

178. Using cancellation tokens in API controllers or actions

- Accept `CancellationToken` parameter in action methods.
 - Pass token to async calls to support request cancellation.
 - Improves responsiveness and resource management.
-

179. What are the default limits for file upload sizes, and how to configure them

- Default max request body size is 30 MB.
 - Configure via `RequestSizeLimit` attribute or `KestrelServerOptions.Limits.MaxRequestBodySize`.
 - For IIS, adjust `maxAllowedContentLength`.
-

180. How to enable gzip or brotli compression



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

- Add `ResponseCompression` middleware.
- Register compression providers (`AddResponseCompression()`) in `Program.cs`.
- Configure supported MIME types and compression level.

181. Working with JSON options: `System.Text.Json` vs `Newtonsoft.Json`

- **`System.Text.Json`**: Default in .NET Core 3+, fast, built-in, less feature-rich.
- **`Newtonsoft.Json`**: More mature, supports advanced scenarios (e.g., polymorphic deserialization).
- Can switch to `Newtonsoft` by adding `AddNewtonsoftJson()` in MVC options.